

Extreme Partial-Sum Quantization for Analog Computing-In-Memory Neural Network Accelerators

YULHWA KIM and HYUNGJUN KIM, Pohang University of Science and Technology, South Korea
JAE-JOON KIM, Seoul National University, South Korea

In Analog Computing-in-Memory (CIM) neural network accelerators, analog-to-digital converters (ADCs) are required to convert the analog partial sums generated from a CIM array to digital values. The overhead from ADCs substantially degrades the energy efficiency of CIM accelerators so that previous works attempted to lower the ADC resolution considering the distribution of the partial sums. Despite the efforts, the required ADC resolution still remains relatively high. In this article, we propose the data-driven partial sum quantization scheme, which exhaustively searches for the optimal quantization range with little computational burden. We also report that analyzing the characteristics of the partial sum distributions at each layer gives an additional information to further reduce the ADC resolution compared to previous works that mostly used the characteristics of the partial sum distributions of the entire network. Based on the finer-level data-driven approach combined with retraining, we present a methodology for extreme partial-sum quantization. Experimental results show that the proposed method can reduce the ADC resolution to 2 to 3 bits for CIFAR-10 dataset, which is the smaller ADC bit resolution than any previous CIM-based NN accelerators.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; • **Hardware** → **Analysis and design of emerging devices and systems**; **Emerging architectures**;

Additional Key Words and Phrases: Computing-in-memory, processing-in-memory, neural networks, analog computing

ACM Reference format:

Yulhwa Kim, Hyungjun Kim, and Jae-Joon Kim. 2022. Extreme Partial-Sum Quantization for Analog Computing-In-Memory Neural Network Accelerators. *J. Emerg. Technol. Comput. Syst.* 18, 4, Article 75 (October 2022), 19 pages.

<https://doi.org/10.1145/3528104>

This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea Government [Ministry of Science and ICT (MSIT)] under Grant 2020-0-01309 (Development of Artificial Intelligence Deep Learning Processor Technology for Complex Transaction Processing Server, 40%) and in part by the National Research Foundation of Korea (NRF) grant funded by the Korea Government (MSIT) under Grant 2020R1A2C2004329 (NeuroHub: Web-based Open Simulation Platform for Collaborative In Memory Neural Network Hardware Research, 50%). This work was also supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT) (Artificial Intelligence Innovation Hub, 10%) under Grant 2021-0-02068. The EDA tool was supported by the IC Design Education Center (IDEC).

Authors' addresses: Y. Kim and H. Kim, Pohang University of Science and Technology, 77, Cheongam-ro, Nam-gu, Pohang, Gyeongsangbuk-do, 37673, South Korea; emails: {yulhwa.kim, hyungjun.kim}@postech.ac.kr; J.-J. Kim (corresponding author), Seoul National University, 1, Gwanak-ro, Gwanak-gu, Seoul, 08826, South Korea; email: kimjaejeon@snu.ac.kr.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1550-4832/2022/10-ART75 \$15.00

<https://doi.org/10.1145/3528104>

1 INTRODUCTION

Processing of **deep neural network (DNN)** requires a large amount of data transfer and computation, so hardware accelerators for efficient DNN processing are being actively studied. Recently, **computing-in-memory (CIM)** hardware is garnering attention as a new computing platform for DNNs [2, 15, 20]. CIM hardware uses memory arrays as both data storage and processing units, thus minimizing data transfer from memory to other processing units. As a result, unlike the conventional Von-Neumann architecture, CIM hardware can avoid memory bottleneck due to the large amount of data transfer.

While CIM accelerators have the potential as an efficient computing platform for DNNs, they meet several challenges due to the nature of analog processing inside the memory array. Since the digital logic is typically used outside the array, **digital-to-analog converter (DAC)** is used to feed input values to analog CIM arrays. The input resolution of CIM arrays is limited to the DAC resolution, and CIM-based NN accelerators need to handle 2- to 8-bit input values to process general NNs [4, 13, 20]. However, area and energy overhead of DAC is proportional to its resolution, and hence multi-bit DACs degrade the area and energy efficiency of CIM accelerators. Therefore, to support bit-scalable computation with low-resolution DAC, most of CIM accelerators process input values in bit-serial manner [8, 15, 20]. In this way, the resolution of DAC can be lowered to 1 bit, and CIM arrays can utilize word-line drivers of typical memory arrays to feed input values.

CIM accelerators also use **analog-to-digital converter (ADC)** to interface the analog **multiplication-accumulation (MAC)** results inside the CIM array to external digital logics. The MAC results typically represent the partial sums. Since partial sums are high-resolution values, CIM accelerators require high-resolution ADCs. The overhead of ADCs also increase as their resolution increases so that ADCs significantly degrade area and energy efficiency of CIM accelerators [2, 15, 20]. Meanwhile, partial sums need to be quantized if low-resolution ADCs are used, and hence there is a great need for efficient and accurate partial-sum quantization algorithm to reduce the ADC overhead.

As a result, several previous works developed partial-sum quantization algorithms for CIM-based NN accelerators [1, 12, 17]. Previous works tried to reduce the **mean-square-error (MSE)** of partial-sum quantization and relied on the min/max values or standard deviation (σ) of the partial sum distributions to decide the quantization range [1, 12, 17]. However, due to non-linearity introduced in NN computation, minimization of MSE does not guarantee the minimization of NN accuracy degradation, and the min/max or the 3σ of partial sum distribution does not represent the quantization range properly. Therefore, previous approaches result in sub-optimal partial-sum quantization results, and higher than 4-bit ADCs are needed to obtain reliable NN accuracy.

In this article, we propose a methodology for more aggressive partial-sum quantization. To design more accurate partial-sum quantization with lower resolution, we first investigate the characteristics of partial sum distributions in bit-scalable CIM hardware. Based on the observation that it is important to quantize the partial sum values near 0 more accurately to reduce NN accuracy loss, we propose a new type of quantizer. Then, the quantizer is further optimized with a data-driven method that finds the better quantization range than previous methods that used 3σ of partial sum distribution. Finally, we demonstrate that retraining NNs with partial-sum quantization recovers significant amount of the NN accuracy loss, so that we can achieve extreme quantization of partial sums.

The key contributions of this article are as follows:

- (1) We propose the zero-tread quantizer that reduces the quantization error near 0 values based on the extensive analysis of partial sum distributions in bit-scalable CIM accelerators.

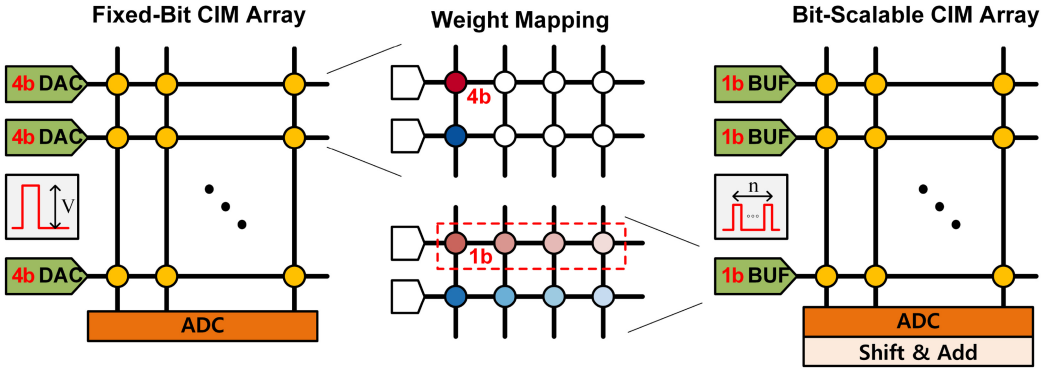


Fig. 1. Analog CIM arrays with fixed-bit computation and bit-scalable computation. This figure shows an example of a 4-bit input and 4-bit weight implementation for a CIM array.

- (2) We optimize the quantization range using a data-driven approach that exhaustively searches for the optimal range from the sampled dataset based on the observation that the search space for the partial sum quantization range is much smaller than the search space for quantization range of other neural network parameters such as weight or activation.
- (3) We evaluate the effectiveness of the proposed algorithm in terms of area and energy efficiency of CIM-based NN accelerators. CIM accelerators co-designed with proposed partial-sum quantization reduce the ADC resolution to 2 to 3 bits for CIFAR-10 dataset so that their area and energy consumption become much smaller than those of previous CIM accelerators.

2 BACKGROUND

2.1 NN Accelerators with Analog CIM Array

Analog CIM accelerators conduct MAC operations on the memory array, and they have potential to reduce massive amount of data movement and enable parallel computations. In the CIM array, the resolution of input and weight values are fixed when the bit resolution of the DAC and a memory cell are decided (Figure 1 (left)) [2]. However, the resolution of input neurons and synaptic weight values may vary depending on the network architecture and target tasks, so general CIM-based NN accelerators need to support bit-scalable computation. To overcome the hardware limitation and support bit-scalable operations, bit-scalable CIM arrays (Figure 1 (right)) have been widely used [15, 20].

In the bit-scalable CIM accelerators, input values are processed in bit-serial manner and weight values are processed in bit-parallel manner. First, rather than mapping input values to multi-level input voltage, bit-scalable CIM arrays receive 0/1 values for n cycles to process an n -bit input. Since the binary values of input bitplanes are fed to CIM arrays at each cycle, the bit-scalable CIM accelerators can replace a n -bit DAC to an 1-bit word-line driver that is used for conventional memory array. Next, weight values are stored in the memory cells, and the bit-parallel mode use multiple cells to represent multi-bit weight values (Figure 1 (middle)). For example, when each memory cell can represent 1-bit value, 4 memory cells are used to represent 4-bit weight value. As different weight bitplanes should be accumulated separately in the analog domain, memory cells for different bitplanes should be allocated in different columns or different CIM arrays.

Although the bit-scalability issue of CIM accelerators are resolved by the bit-serial input/bit-parallel weight computation, still there are some remaining challenges in CIM accelerator design. One of the most important challenges is area and energy overhead caused by peripheral circuits

of CIM arrays [2, 15, 20]. Though CIM accelerators conduct MAC operations in the analog domain, it is difficult to build the entire system in the analog domain. Therefore, ADCs are required between analog CIM array and external digital system. The area and energy consumption of an ADC increases significantly as its resolution increases, so high-resolution ADC seriously affects the overall efficiency of the entire systems [18]. Conventional CIM accelerators adopted bit-serial input processing to eliminate DAC overhead, but ADCs are still required to convert partial sums of analog CIM array and the partial sums are high-resolution values [15, 20]. Thus, it is hard to lower the resolution of ADC, and conventional CIM accelerators use multi-bit ADCs in spite of the overhead. As a result, the overhead of ADCs seriously degrade the energy efficiency of CIM accelerators.

In the fully digital CIM arrays, ADCs are not needed [3] as the MAC operation is conducted with digital circuits. However, the digital-based CIM array requires multiple read (as many times as the number of rows in a array) from the memory array, and it also needs digital MAC operation while an analog-based CIM array enable matrix-vector multiplication with a single read operation. Therefore, the digital approach introduces much more area/energy/latency overhead compared to the analog approach, and there is a great need to lower the resolution of ADC in the analog CIM arrays to achieve full potential of CIM accelerators.

2.2 ADCs and Partial-Sum Quantization

In analog CIM accelerators, ADCs are used to convert analog partial sums generated from each memory array into digital values. The partial sums are the cumulative values of the multiplication result that occurs in each memory cell, so their resolution tends to be high. Let us assume that the row count of the memory array is N_{row} ; we use $[0, 1]$ for binary input values and $[-1, 1]$ for binary weight values as in typical bit-scalable CIM arrays [15, 20]. The possible multiplication results between an input and a weight are $[-1, 0, 1]$; the multiplication results from several cells are accumulated in each column of the array. As the number of memory cell connected to a column is the same as the row count, the possible range of partial sum is $[-N_{row}, +N_{row}]$. Thus, when the row count is 128, the possible range of partial sum is $[-128, 128]$; 8-bit ADCs are required for one-to-one conversion.

Since area and energy consumption of such high-resolution ADCs are much larger than those of other circuit components, the ADCs degrade the efficiency of CIM accelerators significantly. Thus, ADC resolution needs to be lowered to solve these problems, and there are several works tried to lower the ADC resolution. While the naive method requires 8-bit ADCs, ISAAC [15] reduced the resolution by 1 bit by flipping the weights if the sum of weights in a column is large. This method still guarantees one-to-one conversion of the partial sums, but if we further reduce the resolution of ADC without any change in the NN algorithm, then there exists a gap between the analog partial sum values and the digitized partial sum values. The gap is typically called the quantization error. Thus, to lower the resolution of ADC, we need to design an algorithm to reduce the quantization error.

The quantization algorithms can be categorized into two classes: uniform quantization and non-uniform quantization. In the uniform quantization, the intervals between quantization edges are the same (Figure 2(a) (left)). It can be implemented with a typical ADC that generates voltage level of each quantization edge by dividing a single reference voltage V_{ref} with a resistor ladder (Figure 2(a) (right)). However, the non-uniform quantization has no restriction on the size of intervals (Figure 2(b) (left)). This type of quantization adaptively determines the quantization edges taking into account the distribution of partial sums so that it can store more information in the quantized values compared to the uniform quantization. Non-uniform quantization requires special ADCs that can program voltage level of each quantization edge separately (Figure 2(b) (right)).

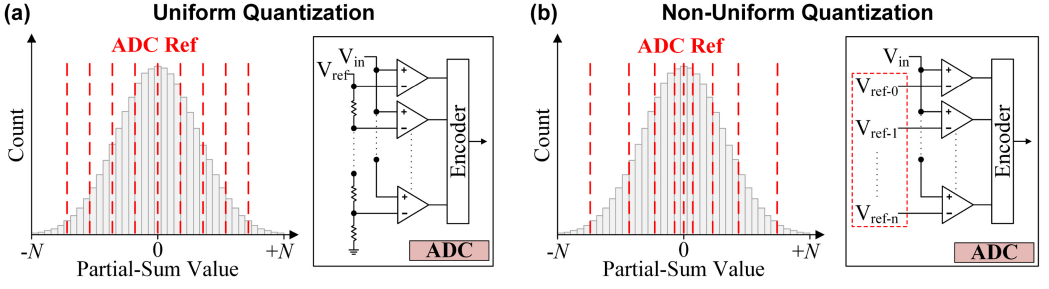


Fig. 2. Partial-sum Quantization and ADC design with (a) Uniform Quantization and (b) Non-Uniform Quantization.

Table 1. Previous Works on Partial-Sum (Psum) Quantization

	Partial-Sum Quantization		ADC Design	
	Quantization Range	Train w/Psum Quant	Type	Resolution
naive	$[-N_{row}, +N_{row}]$	No	Uniform	8 bit
ISAAC [15]	$[-N_{row}, 0]$	No	Uniform	7 bit
LBW [1, 17]	$[min, max]$	Yes	Uniform	6 bit
NeuroSim [12]	Lloyd-Max	No	Non-Uniform	5 bit
EEQ [17]	$[\mu - 3\sigma, \mu + 3\sigma]$	Yes	Non-Uniform	4 bit

The additional circuit components for voltage programming degrade the efficiency of ADCs, so the effective hardware overhead might increase with non-uniform quantization. Thus, our proposed method adopted uniform quantization for lowering ADC resolution.

2.3 Previous Works on Partial-Sum Quantization

There are several previous works that tried to lower ADC resolution with partial-sum quantization [1, 12, 17]. Previous works found that the distribution of the partial sums does not typically cover the theoretical maximum range $[-N_{row}, +N_{row}]$, and they optimized the partial-sum quantizer with various approaches based on the finding. NeuroSim [12] tried to minimize MSE of overall partial sums with Lloyd-Max algorithm. LBW [1] and EEQ [17] defined quantization range, which is the input range of the quantizer, and divided the quantization range with pre-defined rule and given resolution. LBW [1] defined the quantization range with theoretical min/max values of partial sums; EEQ [17] used $[\mu - 3\sigma, \mu + 3\sigma]$ (μ, σ : mean, standard deviation of partial sums) as the quantization range. The methods assumed that the partial sums follow the Gaussian distribution.

We summarized the previous works related to lowering ADC resolution and partial-sum quantization in Table 1. Previous algorithms were mostly evaluated on the relatively easy task, image classification on the CIFAR-10 dataset, and the results show that 4- to 6-bit ADCs are required to maintain NN accuracy. In this article, we analyze the characteristics of partial sums in depth and find that using Gaussian distribution for partial sum has limitation. To overcome the limitation and reduce the ADC resolution more aggressively, we propose a data-driven partial-sum quantization method. In addition, we provide evaluation results on a more complex task, image classification on the ImageNet dataset.

3 DESIGN AND OPTIMIZATION OF PARTIAL-SUM QUANTIZER

3.1 Design of Partial-Sum Quantizer

In this article, we adopt the uniform quantization for the proposed scheme to make it more hardware friendly. The uniform quantizer can be defined with two factors: quantization resolution and

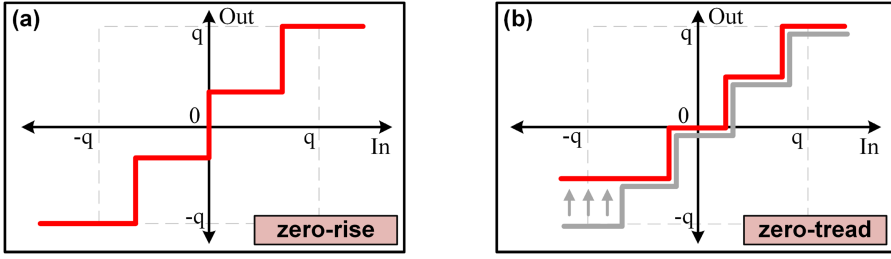


Fig. 3. Two types of uniform quantizers: (a) zero-rise quantizer and (b) zero-tread quantizer. (b) Includes zero in the reconstruction levels.

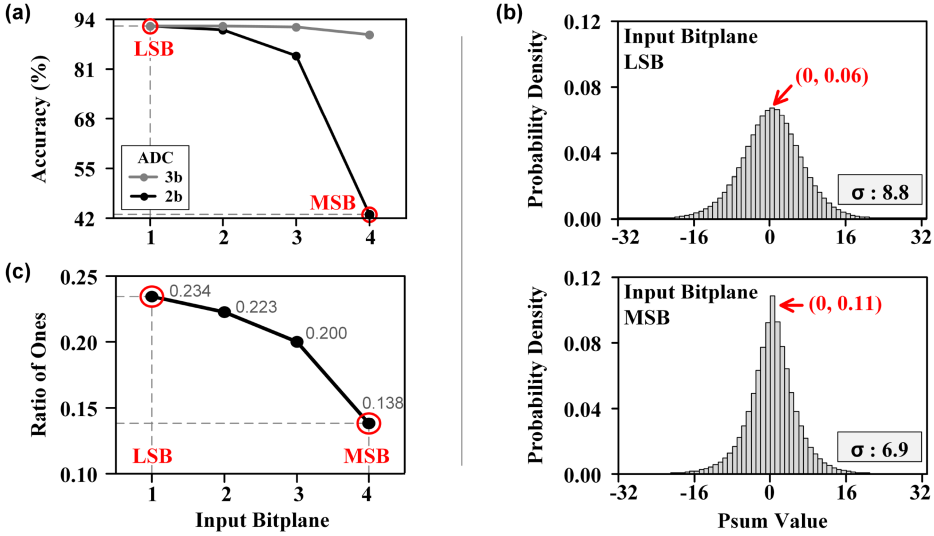


Fig. 4. Properties of partial sums and input bitplane in the bit-serial computation. (a) Classification accuracy with partial-sum quantization on specific input bitplane. Only the partial sums generated from specific input bitplane (x-axis) is quantized. (b) Histogram of partial sums generated from LSB input (top) and generated from MSB input (bottom). (c) The ratio of ones in each input bitplane.

range. The quantization resolution decides the number of quantization intervals, and the quantizer generates each interval by uniformly dividing the quantization range. The overall networkwise distribution of partial sums tends to follow the Gaussian distribution with mean 0 so that the center of the quantization range is generally set to the mean value, 0. Then, the quantization range is symmetric with respect to 0, as the uniform quantizer divides the given quantization range into equal-length intervals (Figure 3(a)). Previous works with uniform quantization used this type of quantizer, and we are going to refer the quantizer as the zero-rise quantizer, because the output value rises at the zero input point. Interestingly, we found that the output change at the zero point has a negative effect on the quantization error because of the dependence of the partial sum distribution on the input bit position.

In the CIM accelerators that perform bit-serial computation, the significance of the analog partial sum depends on the bit significance of input values. To investigate the dependence on input bit position, we quantized the partial sums generated from each input bitplane, and measured the NN accuracies (Figure 4(a)). In the simulation, only the partial sums that are generated from

specific input bitplane are quantized and the partial sums generated from other bitplanes are not quantized. The results show that the partial sum from each bitplane has different sensitivity to the quantization. Partial-sums generated with LSB input showed negligible accuracy change after the quantization while partial-sums with MSB input showed significant accuracy drop. Since partial-sums generated from MSB input are more sensitive to the quantization, we have to put greater importance on MSB cases when we design the quantizer.

To further investigate the characteristics of the partial sums for each bitplane, we compare the distribution of partial sums for LSB and MSB input bitplanes (Figure 4(b)). It is observed that the dispersion of partial sums varies depending on the input bitplane. The variation of the partial sums decreases as the bit significance of the input increases, and the ratio of near-0 values is higher for MSB input than for LSB input. This phenomenon occurs due to different probability of having input value “1” for each input bitplane. Input values are generated by applying ReLU activation function on the MAC values, and the equation of ReLU is as follows:

$$ReLU(x) = \begin{cases} x, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases} \quad (1)$$

Therefore, we can expect that the activation value becomes positive with the probability of ~ 0.5 so that the LSB value of an input activation becomes 1 with the probability of ~ 0.25 . In comparison, the frequency of the large input values is low so that the probability of an MSB value to be 1 is very low. For example, in our experiment for the VGG-9 network, the probability of having positive activation value was 0.417 and that of having the LSB value of 1 was 0.234. Meanwhile, the probability of having the MSB value of 1 was only 0.138 (Figure 4(c)). Thus, when the CIM array processes the MSB of an input, the values fed to the array are mostly 0 and the density of partial sums near 0 becomes higher as shown in Figure 4(b).

Based on the observation, we propose to generate quantization levels that include the value of zero (Figure 3(b)). We are going to refer the proposed quantizer as the zero-tread quantizer in the rest of the article. To implement the zero-tread quantizer, we first add one more levels to the quantization resolution so that the number of conversion levels becomes odd. If we divide the quantization range with the odd number of levels, then we can generate a quantization interval that is flat near zero. Then, we abandon one level in the outer-most interval in the negative region. The zero-tread quantizer assigns one less level to the negative region, but this approach can reduce the quantization error for near 0 input values on which we put more importance.

3.2 Data-driven Optimization of Partial-Sum Quantizer

The uniform quantizer generates quantization intervals by uniformly dividing the quantization range. We can generalize the quantization range as follows:

$$Quantization\ Range = [x - \delta, x + \delta], \quad (2)$$

where x is the center of the quantization range and δ is the bound of the range. The center x is generally set to the mean value of partial sums, which is 0, so the uniform quantizer can be parameterized with δ . The goal of the proposed algorithm is to minimize the NN accuracy loss after partial-sum quantization. Therefore, the optimization of the uniform quantizer is equivalent to finding δ , which results in the minimum NN accuracy loss.

Previous works utilized the iterative method like Lloyd-Max algorithm to minimize the MSE of the partial-sum quantizations or used the min/max values or the 3σ of the overall partial sum distribution for more efficient quantization [1, 12, 17]. However, MAC operation is followed by a non-linear activation function, so the relationship between partial-sum quantization and NN accuracy is not straightforward. Thus, minimization of MSE does not guarantee minimization of

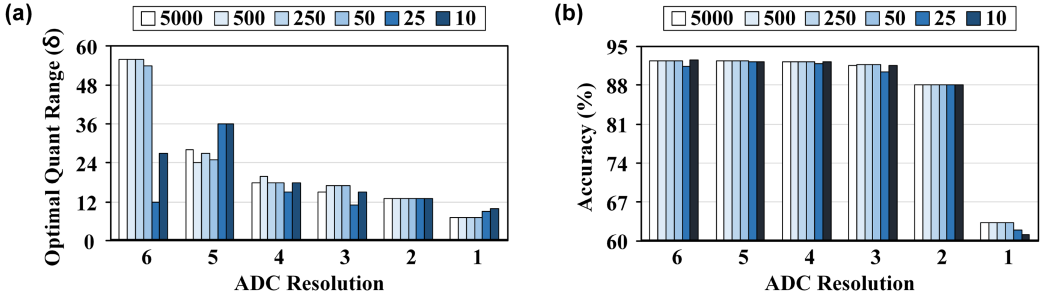


Fig. 5. (a) Optimal quantization range (δ) obtained from data-driven quantization with different image sampling sizes and (b) accuracy of partial-sum quantized VGG-9 with optimal δ . The sampled image count per class varied from 5,000 to 10, and VGG-9 with 4-bit activation and weight was used for the evaluation. For simplicity, only partial sum of the second layer was quantized.

NN accuracy loss. In addition, the min/max values or 3σ do not produce the optimal quantization range. In this article, rather than using statistics of partial sums for the quantization, we adopt the data-driven method to optimize the partial-sum quantizer. The proposed method selects the best δ that can minimize the NN accuracy loss by exhaustively searching over the finite space with the small-size dataset.

Finding the optimal δ for the partial sum quantization has different characteristics from finding the optimal quantization range for other NN parameters. NN parameters such as weight and activation are first trained with **floating-point (FP)** values and then quantized to integer values for efficient NN inference [4, 6]. Due to the high dynamic range of FP values, there are an infinite number of candidates for the quantization ranges for weight and activation values. Hence, it is not possible to find the optimal quantization range using exhaustive search. As a result, the quantizations of weight and activation values are typically done by minimizing the MSE of the parameter distribution or via complex training process [4]. In contrast, partial sums are integer values defined in the narrow range $[-N_{row}, +N_{row}]$ as discussed in Section 2.2, and hence the optimization of the quantizer for the partial sums is a simple task that finds the optimal value of δ from the finite number of the candidates: integers in the range $[1, N_{row}]$. Thus, rather than applying complex training algorithms to find optimal δ , we iteratively scan and evaluate the NN accuracy with each δ value using the sub-training dataset.

In addition, optimization of the partial-sum quantizer does not require as many data as conventional NN training. This intuition is based on two factors. First, the quantization range is represented by a single integer parameter δ only. Second, a single image can generate multiple data samples for the partial sum quantization, because multiple CIM arrays are used to accelerate a single DNN model and each CIM array generates multiple partial sums. Based on the observation, we compare the quality of the data-driven quantization depending on the size of the sampled dataset. From the CIFAR-10 training dataset in which 5,000 images are given for each class, we sampled the equal number of images per class for the experiment. We conducted data-driven quantization with sampled training images and then measured the inference accuracy of the partial-sum quantized model with the test dataset to measure the quality of the quantization. We varied the sampled image count per class from 10 to 5,000 and used VGG-9 as the target DNN model. The partial sums were quantized with 1 to 6 bit resolution. For simplicity, we only quantized the partial sums of the second layer.

As shown in Figure 5, decreasing the number of images per class from 5,000 to 50 did not affect the quantization quality much as both the optimal quantization range and the test accuracy

remained similar. Meanwhile, when the images per class was further decreased to 10–25, the optimal quantization range became different from the other cases and it resulted in up to 2.22% accuracy degradation. For example, when the partial-sum resolution was 1 bit, the optimal quantization range was 7 and the inference accuracy was 63.23% for the cases with 5,000 to 50 images per class; but the optimal range was 9 and 10 and the inference accuracy was 61.83% and 61.01% for the cases with 25 and 10 images per class, respectively. Therefore, based on this experiment, we used 50 images per class for our data-driven quantization.

In the proposed scheme, we sampled 50 images for each class regardless of the target task, so the total size of sampled dataset can differ depending on the target task. For example, the CIFAR-10 dataset has 10 classes, and hence the size of the sampled dataset was 500. However, the sampled dataset of ImageNet has 50,000 images as the ImageNet has 1,000 classes. Since the size of the training dataset was 50,000 and 1,281,167 for CIFAR-10 and ImageNet, respectively, the size of the sub-training dataset is much smaller than that of the entire training dataset in both cases.

3.3 Evaluation of the Zero-Tread Quantizer and the Data-driven Optimization

We apply the proposed algorithm to VGG-9 [16] and ResNet-18 [5] networks with 2,4,8-bit activation and weight values, respectively. We assume that the NNs are implemented on the analog CIM accelerators with array row count 128 and the partial sums are quantized with 2- to 6-bit ADCs. The quantization algorithm is optimized with the proposed data-driven method. We compare the two quantizer types that we discussed in Section 3.1: zero-tread quantizer and zero-rise quantizer. For the evaluation, we measure the classification accuracy with each partial-sum quantizer on the CIFAR-10 dataset. Note that the δ value is selected with sub-training dataset, but the NN accuracy is measured with the original test dataset throughout the article for fair evaluation.

Combination of the zero-tread quantizer and the data-driven optimization algorithm achieves reliable NN accuracy with 3- to 4-bit ADCs that are smaller resolution than that of previous works (Figure 6). The time cost of searching optimal quantization range is varied from 1 minute and 48 seconds to 20 minutes and 42 seconds depending on the bit resolution of the NN parameters as the cost of emulating bit-serial computation of the CIM accelerators is proportional to the bit resolution of NN activation and weight values. For the experiment, we used NVIDIA Titan Xp. Note that the time cost was tolerable in all cases as the data-driven optimization is introduced to inference operation only.

There are two reasons for the improvement. First, the proposed zero-tread quantizer shows great improvement on NN accuracy compared to the conventional zero-rise quantizer. The zero-rise quantizer is symmetric with respect to the mean of partial sum, so the MSE of the zero-rise quantizer is usually smaller than that of the zero-tread quantizer. However, the zero-tread quantizer shows the better NN accuracy than the zero-rise quantizer, which implies that reducing quantization error for input values near 0 is more important than minimizing MSE. Second, previous works defined the quantization range based on the min/max value or 3σ of partial sums, or MSE of the quantization error, but we search for the optimal δ_{opt} from sub-training dataset. Simulation results show that the proposed data-driven method finds the optimal quantization range and provides the smaller NN accuracy loss than the previous partial sum quantization methods (Figure 7).

3.4 Level-based ADC Design

The proposed algorithm lowers the resolution of ADCs below 4 bits. The common design choice for such a low-resolution ADC is the Flash ADC. The Flash ADC typically uses a resistor-ladder to generate reference voltage for each conversion level; comparators are used to compare the analog input and reference voltage levels (Figure 2(a)). In other words, the Flash ADC processes analog-to-digital conversion level by level. For example, when the resolution of ADC is 4 bits

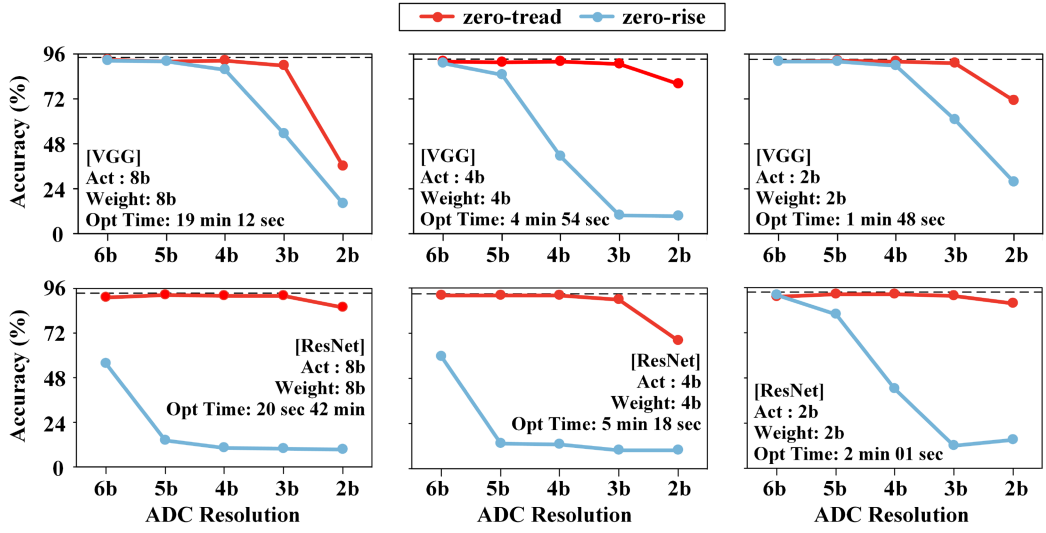


Fig. 6. Classification accuracy of VGG-9 and ResNet-18 with 2- to 6-bit partial-sum quantization. The quantizer for each resolution is optimized with the proposed data-driven method, and two types of quantizer, zero-tread quantizer and zero-rise quantizer, are compared. The dashed-line indicates the accuracy without partial-sum quantization. Opt Time indicates the time cost of searching the optimal quantization range using NVIDIA Titan Xp. (Dataset: CIFAR-10).

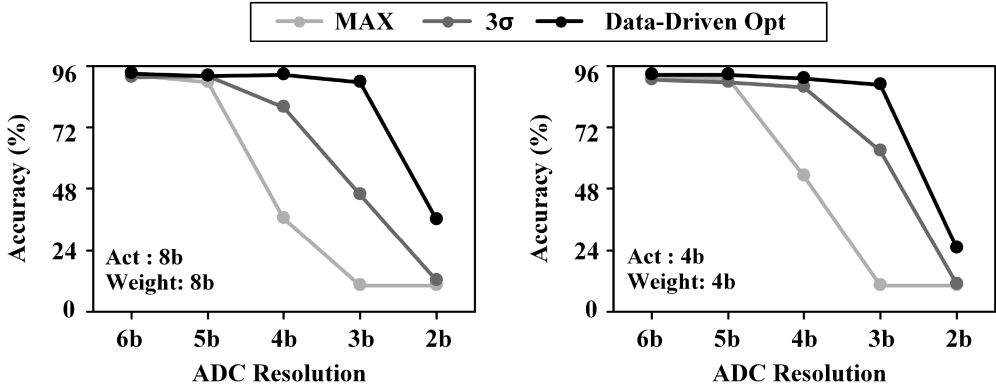


Fig. 7. Classification accuracy of VGG-9 with 2- to 6-bit partial-sum quantization. The quantization range is defined with the min/max value of partial sums, 3σ of partial sums, and the proposed data-driven method. (Dataset: CIFAR-10).

(16 levels), a Flash ADC generates $15(2^4 - 1)$ reference voltage values with resistor-ladder; 15 comparators compare the reference voltages with the input voltage. Meanwhile, if we remove a resistor from the resistor-ladder and a comparator from the ADC, then the Flash ADC generates digital output with 15 levels. Unlike ADCs for general applications, ADCs for CIM readout do not need to have the power-of-2 levels. Controlling the resolution with the number of conversion levels rather than bit resolution allows us to try fine-grained optimization and seek for further area/energy reduction [14, 19]. Therefore, in this work, we try to optimize the number of conversion levels instead of bit resolution.

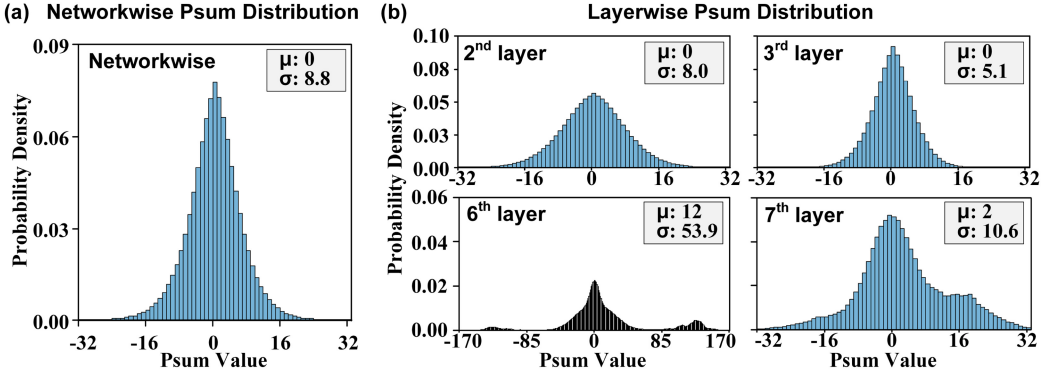


Fig. 8. The breakdown of the partial-sum distribution. (a) The partial-sum distribution over the entire network. (b) The partial-sum distribution of specific layer. (μ : mean of partial sums; σ : standard deviation of partial sums; Network: VGG-9; Dataset: CIFAR-10).

3.5 Layerwise Optimization

To investigate more aggressive methodology for partial-sum quantization, we analyze the distribution of partial sums in networkwise and layerwise manner, respectively (Figure 8). The networkwise partial sum distribution shows the Gaussian-like distribution with a mean of zero, but the distribution of partial sum for each layer varies substantially. The NN layer is the basic unit that comprise NNs. Each layer extracts different information from the input feature maps, and each layer can be biased to extract information. Consequently, the variance and shape of the partial-sum distribution for each layer becomes very different. Previous works and the methodology proposed in the previous section optimized the partial-sum quantizer in a networkwise manner. Therefore, there is a possibility for further improvement if the partial sum quantizer is optimized in a layerwise manner.

When we optimize the quantizer in the networkwise manner, finding the optimal quantization range $[x - \delta, x + \delta]$ is the same as finding optimal δ value, because x is set to 0, the mean of overall partial sums. When we analyze the the partial sum distributions in a layerwise manner, it is observed that the mean of partial sums is not always 0. For example, partial sums for the sixth to seventh layers are not symmetric with respect to 0, so the mean values of these partial sums are not 0 (Figure 8(b)). If we use the mean value as the center of quantization range, then it can reduce MSE of quantization, but it turns out that it does not minimize the NN accuracy loss as we discussed in the previous section.

As both the 0-centered quantizer and the mean-centered quantizer are good candidates for the quantizer design, we compare the NN accuracy of the two quantizers (Figure 9). Here we use the number of conversion levels to represent ADC resolution. The evaluation results show that the 0-centered quantizer achieves the better NN accuracy than the mean-centered quantizer (Figure 9). This result indicates again that near-0 input values are important for the partial sum quantization, so setting the center value of the quantizer to be 0 is effective regardless of the mean values.

We also compare the networkwise and layerwise partial-sum quantizations using AlexNet [10] for ImageNet dataset to evaluate for the larger NN and dataset. We assume that the NN is implemented on the CIM accelerators with array row count 256. The network uses 4-bit activation and weight values. As both Top-1 accuracy and Top-5 accuracy are frequently used for evaluating the quality of the ImageNet classification, we report both metrics after the partial-sum quantization

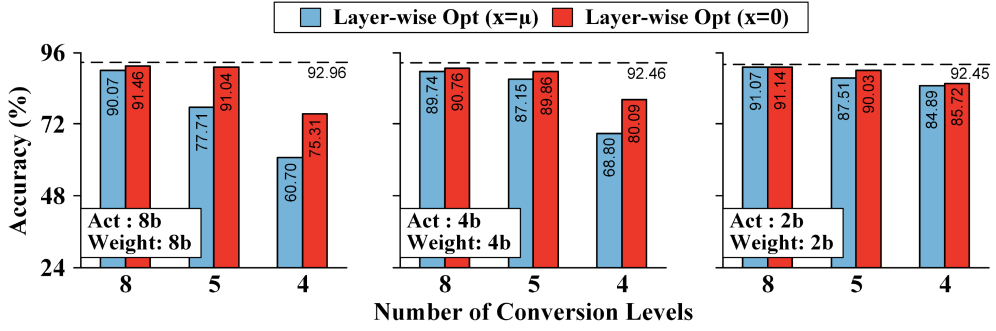


Fig. 9. Classification accuracy with partial-sum quantization. The quantizer is optimized in a layerwise manner. Center (x) of the quantization range is set to mean (μ) of layer partial sums or 0 for the comparison. The dashed line indicates the accuracy without partial-sum quantization (Network: VGG-9; Dataset: CIFAR-10).

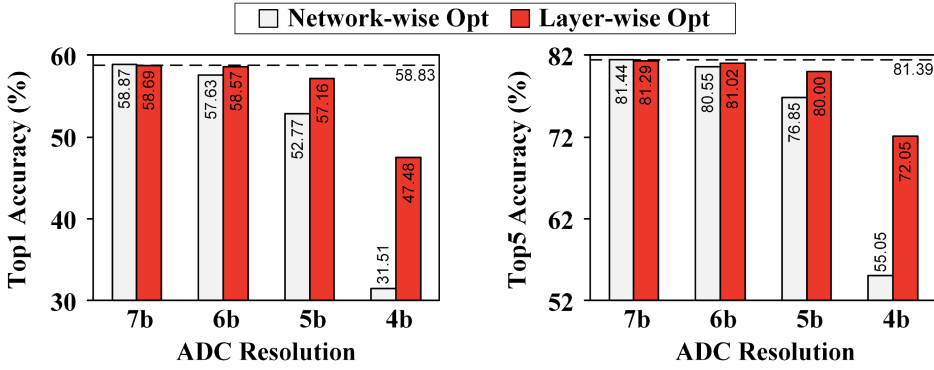


Fig. 10. Classification accuracy with partial-sum quantization. The quantizer is optimized in a networkwise manner and layerwise manner. The dashed-line indicates the accuracy without partial-sum quantization (Network: AlexNet; Dataset: ImageNet).

(Figure 10). Since ImageNet classification is more complex, the network is more sensitive to the partial-sum quantization than the network for CIFAR-10 classification. To prevent accuracy drop, 7-bit ADCs are required with networkwise partial-sum quantizer; 6-bit ADCs are required with a layerwise quantizer. Even 5-bit ADCs produce reasonable accuracy when the layerwise quantizer is used. In summary, the required ADC resolution for the layerwise quantizer is 1 to 2 bits lower than that for a networkwise quantizer. Note that 1-bit resolution reduction halves the area and energy consumption of ADC, so the 1- to 2-bit reduction of ADC resolution is meaningful.

In the layerwise optimization, the optimum quantization range for each layer can be different, and the quantization range decides the reference voltage of the flash ADC (Figure 2(a)). Therefore, to implement the layerwise scheme on the CIM accelerators, the reference voltage of the ADC should be adaptively controlled. For the hardware implementation, we can use either external variable V_{ref} or a dummy column to encode the reference voltage as proposed in Reference [9], in which an additional column is added to each CIM array to drive the reference voltage. The area and energy overhead of this implementation is negligible, because multiple columns share a single dummy column. For example, in the CIM array with size 256×256 , 256 columns share a single dummy column and the dummy column accounts for 0.39% of the original array.

4 EXTREME PARTIAL-SUM QUANTIZATION WITH RETRAINING

4.1 Retraining of NN Parameters

In the previous section, we proposed the methods to design and optimize the partial-sum quantizer. Distortion in the partial-sum quantization was reduced by the proposed methodologies, and the ADC resolution required to prevent NN accuracy loss was significantly reduced. The proposed methods optimized the quantizer without training of NN parameters, so the optimization process did not need heavy computational burden. However, as the resolution of the quantizer decreases, there is a fundamental limit to reduce the quantization error. Therefore, to achieve extreme partial-sum quantization, the NN parameters need to be retrained so that the network can adapt to the new constraints in terms of the partial sum resolution.

In the retraining stage, the partial-sum quantizer optimized with the proposed data-driven approach is applied to the NN model, and the NN parameters such as synaptic weights are trained to adjust the NN model to recover the accuracy loss due to the partial-sum quantization. As the term “retraining” indicates, we do not train the parameters from scratch and instead initiate the training process with the pre-trained parameters. For example, when mapping VGG-9 with 4-bit activation and weight values to the CIM accelerators, we first adopt the VGG-9 model pre-trained with conventional activation and weight quantization techniques [6]. We optimize the partial-sum quantization on this model to reduce the ADC resolution for the CIM implementation, and then the NN parameters of VGG-9 are retrained aware of partial-sum quantization. As we do not have to train the parameters from scratch, the retraining process can save the training iterations, and reducing training iteration is important for training with partial-sum quantization. For example, training the VGG-9 model from scratch required 120 training epochs, but retraining with partial-sum required only 25 training epochs.

Contrary to the previous section in which NN parameters were fixed and the partial-sum quantizer was optimized, the partial-sum quantizer is fixed, and the NN parameters are retrained in this section. NN parameters can be categorized into synaptic weights used for feature extraction and batch-normalization parameters used for balancing neuron values, and training them has two major effects on recovering the NN accuracy loss. First, fine-tuning the synaptic weights reconstructs the NNs to extract proper information. The partial-sum quantization changes the NN computation results, so the features that are extracted from the NNs are also changed. However, if we fine-tune the synaptic weights that are the filters for the feature extraction, then NNs can adapt to the computational change and extract the desired information again. Second, retraining the batch-normalization parameters can reduce the quantization error by fine-tuning the reconstruction level of quantization. The reconstruction level is the magnitude of quantization output (y -axis of the Figure 3). The batch-normalization parameters define the scale-and-shift operation of MAC results [7], so the quantization output is scaled with batch-norm parameters and the effective reconstruction level can be fine-tuned by retraining.

The retraining uses the back-propagation similarly to the conventional NN training [11]. The learning rate of the fine-tuning is set to be smaller than the learning rate used for the baseline NN training without partial-sum quantization. The other training conditions are the same as those for training the baseline NNs. Simulation results show that the retraining recovers the NN accuracy loss from the partial-sum quantization significantly (Figure 11). While the partial-sum quantization with data-driven optimization required five- to eight-level ADCs to preserve NN accuracy, the partial-sum quantization with retraining requires three-level ADCs to achieve reliable NN accuracy for CIFAR-10 dataset. Therefore, though it requires additional computational burden, retraining has noticeable advantage in reducing ADC resolution.

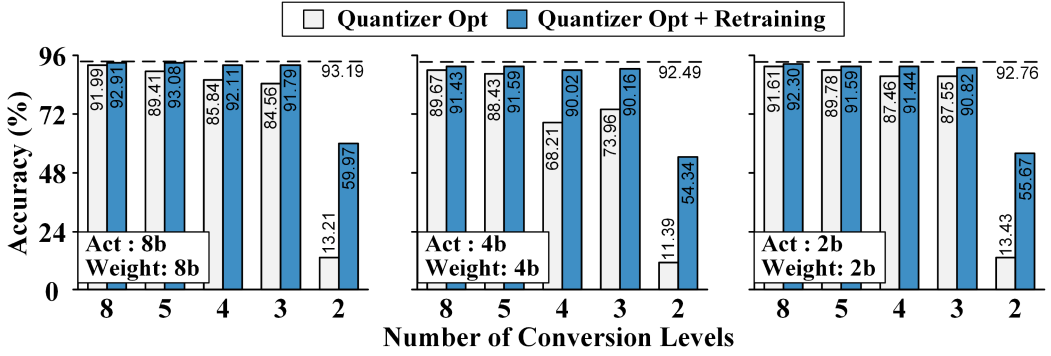


Fig. 11. Classification accuracy with partial-sum quantization. The partial-sum quantizer is optimized with data-driven method, and NN parameters are retrained with partial-sum quantization. The dashed line indicates the accuracy without partial-sum quantization (Network: ResNet-18; Dataset: CIFAR-10).

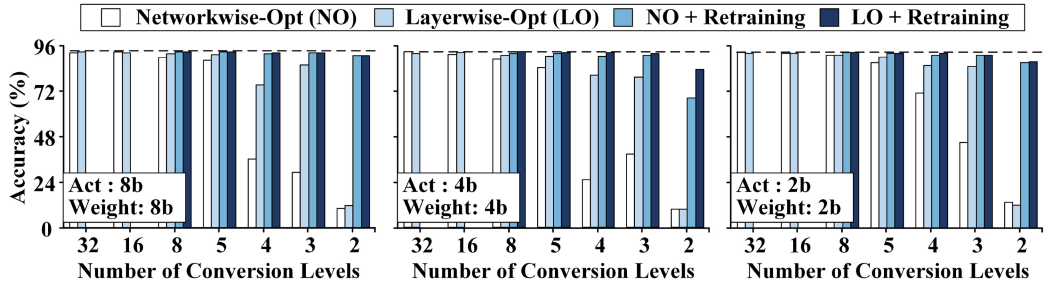


Fig. 12. Classification accuracy with proposed partial-sum quantization. The dashed line indicates the accuracy without partial-sum quantization (Network: VGG-9; Dataset: CIFAR-10).

4.2 Impact of Data-driven Optimization and Retraining

In this section, we evaluate the impact of the proposed data-driven partial-sum quantization and retraining methodologies in terms of NN accuracy and ADC resolution.

First, we evaluate the impact of the proposed method on the CIFAR-10 dataset (Figure 12). When we optimize the quantization function with a data-driven approach, partial-sum quantization with layerwise optimization shows the better NN accuracies than the networkwise optimization, so the required ADC resolution with the layerwise optimization is less than that of the networkwise optimization. The layerwise optimization preserves the NN accuracy with five-level ADCs, while the networkwise optimization preserves the NN accuracy with eight-level ADCs. Retraining NN parameters with partial-sum quantization fully recovers the NN accuracy loss except for two-level ADC (1-bit ADC), so the networkwise optimization and layerwise optimization show the same level of NN accuracy after retraining.

Next, we evaluate the impact of proposed method on the larger dataset, the ImageNet dataset (Figure 13). NNs for ImageNet classification are more sensitive to partial sum quantization than NNs for CIFAR-10 classification, so that even the layerwise optimization requires 32-level ADC (5-bit ADC) to prevent noticeable NN accuracy loss (Figure 10). However, the retraining process effectively recovers the accuracy loss and reduces the required ADC resolution. Similarly to the CIFAR-10 case, the partial-sum quantization with layerwise optimization tends to achieve the better NN accuracy than the quantization with the networkwise optimization. In summary,

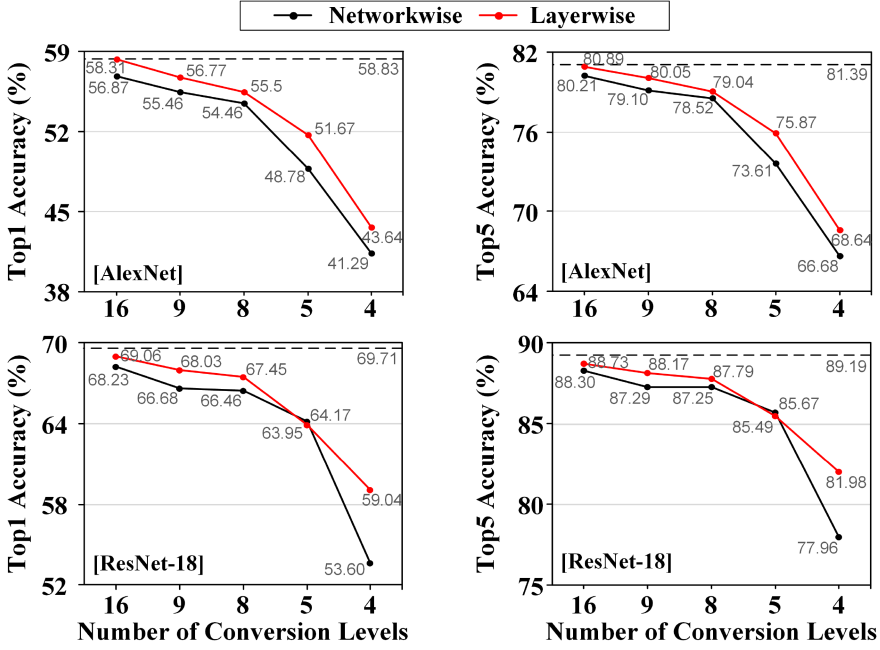


Fig. 13. Classification accuracy with partial-sum quantization. After the partial-sum quantizers are optimized with data-driven method, NNs are retrained with partial-sum quantization. The dashed line indicates the accuracy without partial-sum quantization (Dataset: ImageNet).

combination of the layerwise partial sum quantization and retraining produces the best results for ImageNet dataset; the ADC resolution can be reduced to 9–16 levels (≤ 4 bit).

Finally, we compare the proposed method with previous works [1, 12, 17, 20] (Table 2). The proposed methods preserved inference accuracy only with 3-to 4-level ADCs on the CIFAR-10 dataset, and 9- to 16-level ADCs were required to preserve inference accuracy on ImageNet dataset (Table 2). Note that previous works required more than 16-level ADCs even on the CIFAR-10 dataset, and to the best of our knowledge, there are no previous works that reported partial sum quantization results on ImageNet dataset.

5 HARDWARE EVALUATION

5.1 Hardware Evaluation Setups

In this section, we evaluate the area and energy efficiency of CIM accelerators co-designed with the proposed partial sum quantization. The proposed scheme lowers the ADC resolution with extreme partial-sum quantization, and CIM accelerators with different ADC resolution are evaluated to assess the impact of proposed methods compared to previous methods. We use NeuroSim [12], a benchmark framework of CIM accelerators, to measure the area and energy efficiency of CIM accelerators. The evaluation is focused on the memory array, input drivers, and read-out circuits. One-bit word-line drivers are used as input drivers and a couple of 1T1R RRAM cells are used to represent a signed weight [20].

The evaluation use multiple sense-amp based flash-type ADC. The major components of the flash-type ADC are comparators (Figure 2(a)), so we design a sense amplifier for CIM and the sense amplifiers are used as the comparators for the flash-type ADC. In the flash-type ADC, as

Table 2. Comparison in Previous Partial-Sum Quantization Methods and the Proposed Method

	On CIFAR-10 dataset				On ImageNet dataset			
	VGG		ResNet		AlexNet		ResNet	
	Top1 Loss (%)	ADC level	Top1 Loss (%)	ADC level	Top1 Loss (%)	ADC level	Top1 Loss (%)	ADC level
LBW [1, 17]	0.93	64	1.31	64	—	—	—	—
CMP [20]	0.50	64	0.42	64	—	—	—	—
NeuroSim [12]	0.68	32	—	—	—	—	—	—
EEQ [17]	0.50	16	2.32	16	—	—	—	—
Proposed	0.92	4	1.08	4	0.52	16	0.65	16
	0.87	3	1.40	3	2.06	9	1.68	9

Top1 loss indicates Top1 accuracy drop of partial-sum quantized NN models compared to the original NN models.

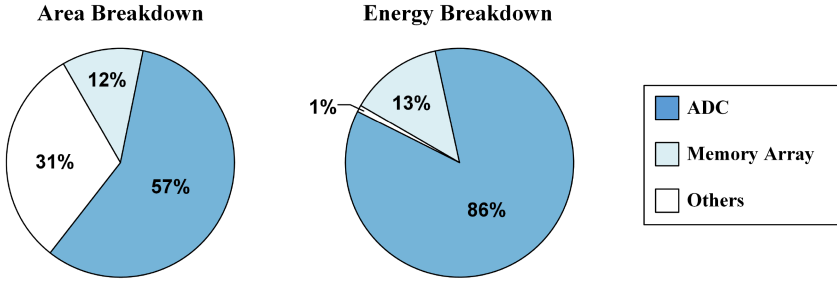


Fig. 14. Breakdown of area and energy consumption of a previous CIM hardware [17]. The size of memory array is 128×128 , and 16-level ADC (4-bit ADC) is used.

the number of the comparators is proportional to the number of ADC conversion level, the area and energy of flash-type ADC is proportional to the ADC level.

All the circuit components are evaluated in a 32-nm CMOS technology. We test the proposed methods on two types of NN models: VGG-9 [16] and AlexNet [10]. VGG-9 is evaluated on CIFAR-10 dataset and the CIM implementation for it uses a 128×128 memory array; AlexNet is evaluated on the ImageNet dataset, and the CIM implementation uses 256×256 memory array.

5.2 Breakdown of Area and Energy Consumption

Before we evaluate the CIM accelerators co-designed with the proposed scheme, we first evaluate the area and energy consumption of an analog CIM accelerator based on previous partial sum quantization methods as a baseline [17]. The accelerator uses the memory array with size 128×128 and 16-level ADCs (4-bit ADCs), and the analysis results are summarized in Figure 14. In the CIM accelerator, ADCs consume most of area and energy; ADCs account for 57% of total area and 86% of energy consumption. This result implies that the overhead of ADC is significant in CIM-based NN accelerators, so lowering ADC overhead can improve overall efficiency of the CIM accelerators.

5.3 Area Efficiency

Previous works used ADCs with more than 16 conversion levels to implement NNs trained on CIFAR-10 dataset [1, 12, 17], but the proposed method requires 5-to 8-level ADCs thanks to the data-driven optimization and only 3-level ADCs when retraining is added. Therefore, we evaluate the total area of a CIM accelerator using 3- to 16-level ADCs (Figure 15(a)). The area of ADC decreases as its resolution is lowered, because the number of comparators and resistors in the resistor-ladder reduces as the number of conversion level decreases. The CIM accelerator with

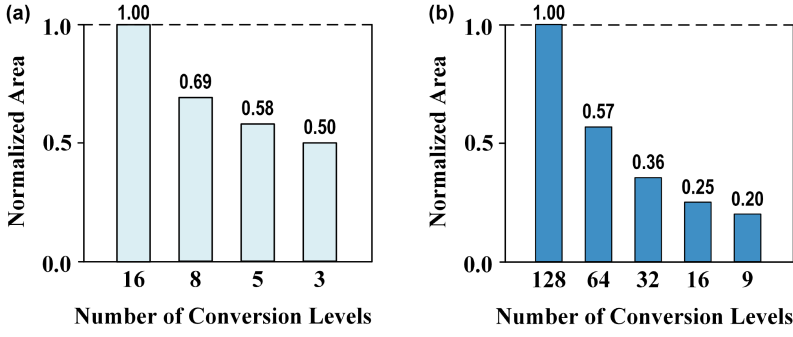


Fig. 15. Area of CIM hardware with various ADC resolution. CIM hardware for (a) VGG-9 and (b) AlexNet. The area is normalized with the maximum value for each design.

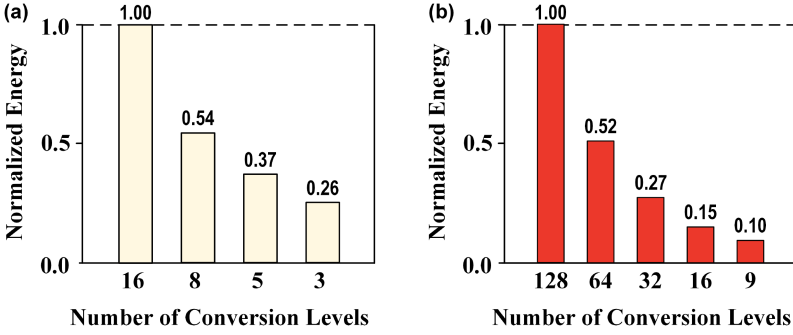


Fig. 16. Energy consumption of CIM hardware processing (a) VGG-9 and (b) AlexNet. The energy consumption is measured with various ADC resolution, and the area is normalized with maximum value.

8-level ADCs saves 31% of total area compared to the baseline, which uses the 16-level ADC; the CIM accelerator with 3-level ADCs saves 50% of total area compared to the baseline.

Since there is no available reference to the ADC overhead reduction in CIM for ImageNet classification, a CIM accelerator co-designed with networkwise optimization is used as the baseline. NNs for ImageNet classification require higher ADC resolution than NNs for CIFAR-10 classification. The data-driven δ searching with networkwise optimization requires 128-level ADC (7-bit ADC). The ADC resolution is lowered to 32-to 64-level ADC (5-6 bit ADC) with layerwise optimization. The layerwise optimization reduced ADC resolution by 64–96 levels compared to the networkwise optimization, and it saves 43–64% of total area (Figure 15(b)). The retraining reduces the ADC resolution to 9 to 16 levels. In this case, total area is saved up to 80%. The reduction of total area in ImageNet case is larger than the reduction in CIFAR-10 case, because the impact of reducing the resolution becomes larger when the baseline ADC resolution is high.

5.4 Energy Efficiency

The energy consumption of analog CIM accelerators is also reduced as the resolution of ADC is lowered. The CIM accelerators based on the proposed schemes save 46–74% energy consumption for CIFAR-10 classification by reducing ADC resolution from 16 conversion levels to 3 conversion levels; CIM accelerators for ImageNet classification save 48–90% energy consumption. The reduction in the energy consumption is larger than the reduction in the total area, because multiple columns of CIM array share an ADC. The ADC sharing reduces the required number of ADCs, so the area overhead of ADCs can be reduced. However, the energy consumption of ADCs is

proportional to the number of conversion, and ADC sharing does not change the number of conversion. Thus, ADC sharing does not reduce energy consumption of ADCs, and hence ADC overhead is more significant in energy consumption than the total area (Figure 16). As a result, reducing ADC overhead has a greater impact on energy efficiency than area.

6 CONCLUSION

In this article, we proposed an aggressive partial-sum quantization methodology to reduce ADC overhead in CIM-based NN accelerators. The proposed method was developed on the bit-scalable CIM hardware to support generic multi-bit NN computations. We first introduced the zero-tread quantizer to reduce the quantization error for near 0 values, and the partial-sum quantizer is optimized with a data-driven approach. The partial sum was optimized in networkwise manner or layerwise manner, and the layerwise optimization showed better NN accuracy. Last, we retrained the NNs with partial-sum quantization, which recovered substantial loss of the NN accuracy caused by partial-sum quantizations, so that extreme quantization of partial sums could be achieved. Evaluation results show that NN accuracy on the CIFAR-10 dataset is preserved with three-level ADC, and NN accuracy on the ImageNet dataset is preserved with nine-level ADC. As a result, CIM accelerators that are co-designed with proposed quantization algorithm achieve 50–80% reduction in total area and 74–90% reduction in the energy consumption compared to previous designs.

REFERENCES

- [1] Yi Cai, Tianqi Tang, Lixue Xia, Boxun Li, Yu Wang, and Huazhong Yang. 2020. Low bit-width convolutional neural network on RRAM. *IEEE Trans. Comput.-Aid. Des. Integr. Circ. Syst.* 39, 7 (2020), 1414–1427.
- [2] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory. In *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA'16)*. IEEE, 27–39.
- [3] Yu-Der Chih, Po-Hao Lee, Hidehiro Fujiwara, Yi-Chun Shih, Chia-Fu Lee, Rawan Naoos, Yu-Lin Chen, Chieh-Pu Lo, Cheng-Han Lu, Haruki Mori, Wei-Chang Zhao, Dar Sun, Mahmut E. Sinangil, Yen-Huei Chen, Tan-Li Chou, Kerem Akarvardar, Hung-Jen Liao, Yih Wang, Meng-Fan Chang, and Tsung-Yung Jonathan Chang. 2021. 16.4 An 89TOPS/W and 16.3TOPS/mm² all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications. In *Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC'21)*.
- [4] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. Pact: Parameterized clipping activation for quantized neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR'18)*.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'16)*. 770–778.
- [6] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *J. Mach. Learn. Res.* 18, 1 (2017), 6869–6898.
- [7] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML'15)*.
- [8] Hongyang Jia, Murat Ozatay, Yinqi Tang, Hossein Valavi, Rakshit Pathak, Jinseok Lee, and Naveen Verma. 2021. A programmable neural-network inference accelerator based on scalable in-memory computing. In *Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC'21)*. IEEE, 236–238.
- [9] Jinseok Kim, Jongeun Koo, Taesu Kim, Yulhwa Kim, Hyungjun Kim, Seunghyun Yoo, and Jae-Joon Kim. 2019. Area-efficient and variation-tolerant in-memory BNN computing using 6T SRAM array. In *Symposium on VLSI Circuits*.
- [10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS'12)*.
- [11] Yann LeCun, Bernhard Boser, John S. Denker, Donnie Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. 1989. Backpropagation applied to handwritten zip code recognition. *Neural Comput.* 1, 4 (1989), 541–551.
- [12] Xiaochen Peng, Shanshi Huang, Yandong Luo, Xiaoyu Sun, and Shimeng Yu. 2019. DNN+NeuroSim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies. In *Proceedings of the IEEE International Electron Devices Meeting (IEDM'19)*. IEEE, 32.5.1–32.5.4.

- [13] Sungju Ryu, Hyungjun Kim, Wooseok Yi, Jongeun Koo, Eunhwan Kim, Yulhwa Kim, Taesu Kim, and Jae-Joon Kim. 2020. A 44.1 TOPS/W precision-scalable hardware accelerator for quantized neural networks in 28nm CMOS. In *Proceedings of the IEEE Custom Integrated Circuits Conference (CICC'20)*. IEEE, 1–4.
- [14] Jyotishman Saikia, Shihui Yin, Sai Kiran Cherupally, Bo Zhang, Jian Meng, Mingoo Seok, and Jae sun Seo. 2021. Modeling and optimization of SRAM-based in-memory computing hardware design. In *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE'21)*.
- [15] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *Proceedings of the ACM/IEEE International Symposium on Computer Architecture (ISCA'16)*. IEEE, 18–22.
- [16] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the International Conference on Learning Representations (ICLR'15)*.
- [17] Hanbo Sun, Zhenhua Zhu, Yi Cai, Xiaoming Chen, Yu Wang, and Huazhong Yang. 2020. An energy-efficient quantized and regularized training framework for processing-in-memory accelerators. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC'20)*. IEEE, 325–330.
- [18] Qian Wang, Yongtae Kim, and Peng Li. 2016. Neuromorphic processors with memristive synapses: Synaptic interface and architectural exploration. *ACM J. Emerg. Technol. Comput. Syst.* 12, 4 (2016), 1–22.
- [19] Shihui Yin, Zhewei Jiang, Jae-Sun Seo, and Mingoo Seok. 2020. XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks. *IEEE J. Solid-State Circ.* 55, 6 (2020), 1733–1743.
- [20] Zhenhua Zhu, Hanbo Sun, Yujun Lin, Guohao Dai, Lixue Xia, Song Han, Yu Wang, and Huazhong Yang. 2019. Configurable multi-precision CNN computing framework based on single bit RRAM. In *Proceedings of the ACM/IEEE Design Automation Conference (DAC'19)*. IEEE, 1–6.

Received July 2021; revised January 2022; accepted March 2022